Advanced Computer Architecture CMP 511

Principles of Pipelining and Vector Processing

Department of Computer Engineering

Pokhara University

REDUCED INSTRUCTION SET COMPUTERS: RISC

- **Characteristics of RISC:**
  - Relatively few instructions
  - Rigid instruction format - Fixed-length, easily decoded instruction format
  - Relatively few addressing modes
  - Memory access limited to load and store instructions
  - All operations done within the registers of the CPU
  - Instruction take only one cycle time
  - Complexity in the complier (harder to program)
  - Hardwired rather than micro programmed control
  - RISC pipelining - pipelining is easier
  - Eg. MIPS, ARM Processor, RISC-V

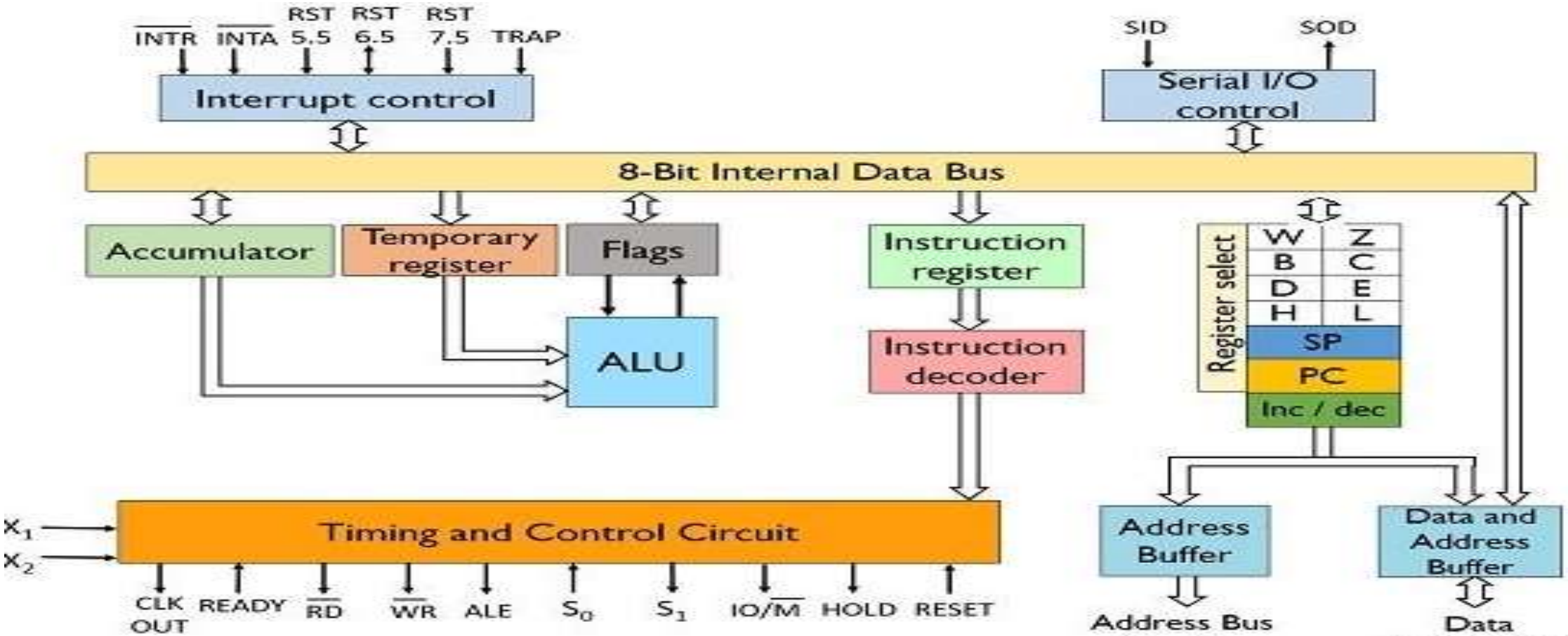
REDUCED INSTRUCTION SET COMPUTERS: RISC

- **More RISC characteristics**
  - A relatively large numbers of registers in the processor unit.
  - Efficient instruction pipeline
  - Compiler support: provides efficient translation of high-level Language programs into machine language programs
- **Advantages of RISC:**
  - VLSI Realization
  - Computing Speed
  - Design Costs and Reliability
  - High Level Language Support

COMPLEX INSTRUCTION SET COMPUTERS: CISC

- High Performance General Purpose Instructions
- **Characteristics of CISC:**
  1. A large number of instructions (from 100-250 usually)
  2. Some instructions that performs a certain tasks are not used frequently.
  3. Many addressing modes are used (5 to 20)
  4. Variable length instruction format.
  5. Instructions that manipulate operands in memory
  6. Micro programmed control unit
  7. CISC pipelining – poor pipelining

Internal working of a processor



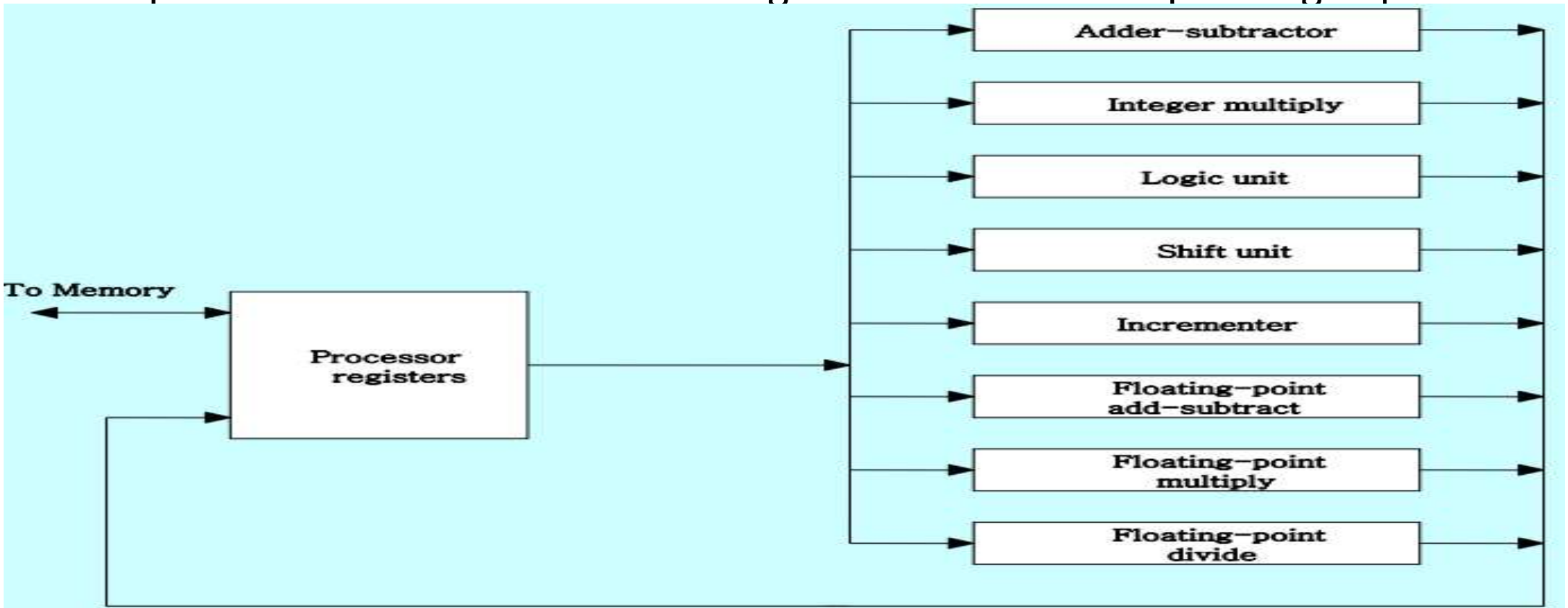
Architecture of 8085 Microprocessor

Parallel Processing

- It is a technique to provide simultaneous data processing task
- In conventional processing system, a single instruction gets executed between the processor and main memory for only one time
- But if the same task/instruction is executed by the number of processors at the same time, then it is known as parallel processing
- The system may have two or more ALUs to be able to execute two or more instruction at the same time.
- The system may have two or more processors operating concurrently.
- It can be achieved by having multiple functional units that perform same or different operation simultaneously.

Process with multiple functional units

- Multiple functional unit
  - In order achieve parallel processing a typical process with multiple functional units are given below; just as how the communication between the body and brain exists.
  - Separates the execution unit into eight functional units operating in parallel



PARALLEL COMPUTERS

Architectural Classification

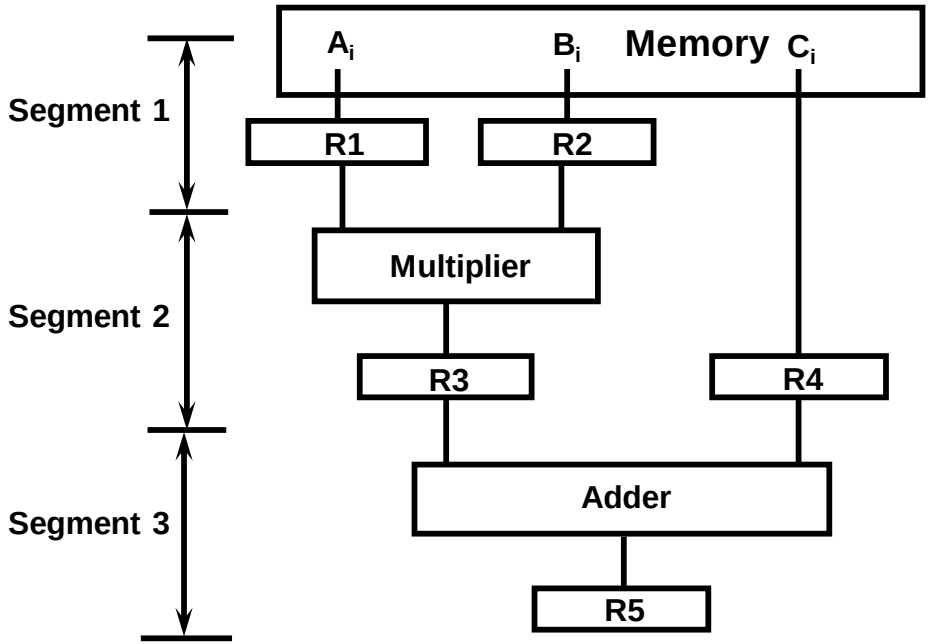
- **Flynn's classification**
  - Based on the multiplicity of *Instruction Streams* and *Data Streams*
  - Instruction Stream
    - Sequence of Instructions read from memory
  - Data Stream
    - Operations performed on the data in the processor

		Number of *Data Streams*	
		Single	Multiple
Number of *Instruction Streams*	Single	SISD	SIMD
	Multiple	MISD	MIMD

PIPELINING

A technique of decomposing a sequential process into sub operations, with each sub process being executed in a partial dedicated segment that operates concurrently with all other segments.

$A_i * B_i + C_i$ for i = 1, 2, 3, ... , 7



$R1 \leftarrow A_i$, $R2 \leftarrow B_i$	Load A_i and B_i
$R3 \leftarrow R1 * R2$, $R4 \leftarrow C_i$	Multiply and load C_i
$R5 \leftarrow R3 + R4$	Add

OPERATIONS IN EACH PIPELINE STAGE

TABLE Content of Registers in Pipeline Example

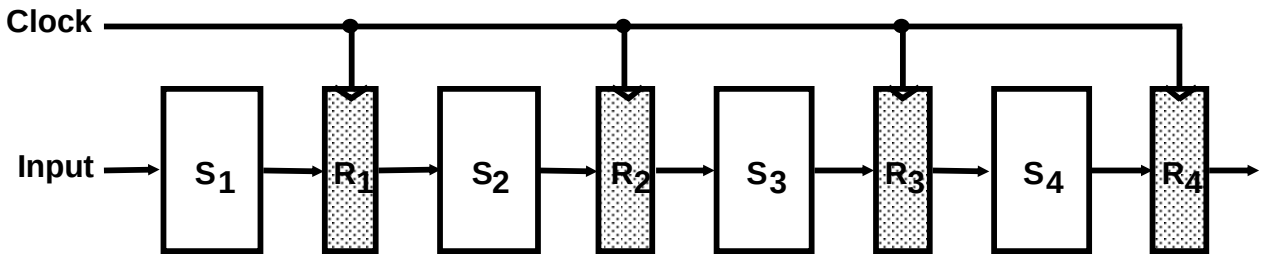
Clock Pulse Number	Segment 1		Segment 2		Segment 3
	$R1$	$R2$	$R3$	$R4$	$R5$
1	A_1	B_1	—	—	—
2	A_2	B_2	$A_1 * B_1$	C_1	—
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	—	—	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	—	—	—	—	$A_7 * B_7 + C_7$

GENERAL PIPELINE

- Pipelining is just the management of the hardware elements of the CPU such that the overall performance is increased
- Simultaneous execution of one or more than one instructions takes place in pipelined processors
- In pipelining, multiple overall instructions are overlapped in execution

GENERAL PIPELINE

General Structure of a 4-Segment Pipeline



Space-Time Diagram : shows segment utilization as the function of time

Segment	1	2	3	4	5	6	7	8	9
									Clock cycles
1	T1	T2	T3	T4	T5	T6			
2		T1	T2	T3	T4	T5	T6		
3			T1	T2	T3	T4	T5	T6	
4				T1	T2	T3	T4	T5	T6

Task : T1, T2, T3,…,T6 executed in four segments.

PIPELINE SPEEDUP

n: Number of tasks to be performed

Conventional Machine (Non-Pipelined)

t_n: Clock cycle

τ_1: Time required to complete the n tasks

$\tau_1 = n * t_n$

Pipelined Machine (k stages)

t_p: Clock cycle (time to complete each sub-operation)

τ_k: Time required to complete the n tasks

$\tau_k = (k + n - 1) * t_p$

Speedup

S: Speedup

$S = n*t_n / (k + n - 1)*t_p$

If 'n' becomes larger then K+n-1 approaches the value of 'n'.

$$\lim_{n \to \infty} S = \frac{t_n}{t_p}$$

PIPELINE AND MULTIPLE FUNCTION UNITS

Example

- 4-stage pipeline
- sub-operation in each stage; t_p = 20ns
- 100 tasks to be executed
- 1 task in non-pipelined system; 20*4 = 80ns

Pipelined System

$(k + n - 1)*t_p = (4 + 99) * 20 = 2060ns$

Non-Pipelined System

$n*k*t_p = 100 * 80 = 8000ns$

Speedup

S = 8000 / 2060 = 3.88

Solve: A 4 stage pipeline has stage delay as 150, 120, 160 and 140 ns. Registers are used in between stages and have a delay of 5 ns. Assuming constant clock rate;

Find the total time to process 1000 data items on this pipeline

Hint: Speed up ratio (S) = $\frac{nt_n}{[K+(n-1)]*tp}$

$$= \frac{total\ time\ required\ for\ n-task\ in\ non\ pipeline}{total\ time\ required\ for\ n-task\ in\ pipeline}$$

As the result of task increases, n becomes much larger resulting to n-1 to be n.

So, K+(n-1) => n

Hereby, maximum speed up ratio (S) = $\frac{ntn}{ntp} = \frac{t_n}{tp}$

where,

k = no of segments

t_p = time required for n task in pipeline

n = no of task/instructions

t_n = time required for n task in non pipeline

Solve

- Consider a non-pipelined processor with a clock rate of 2.5 GHz and average cycles/instruction of 4. The same processor is upgraded to a pipelined processor with 5 stages, but due to internal pipeline delay, the clock speed is reduced to 2 GHz. Assume that there is no stall in pipeline, Calculate the speed up achieved in pipeline processor

ARITHMETIC PIPELINE

Floating-point adder

$X = A \times 2^a$
$Y = B \times 2^b$

[1] Compare the exponents
[2] Align the mantissa
[3] Add/sub the mantissa
[4] Normalize the result

$X=0.9504x10^3$
$Y=0.8200x10^2$

1.The two exponents are subtracted in the first segment.

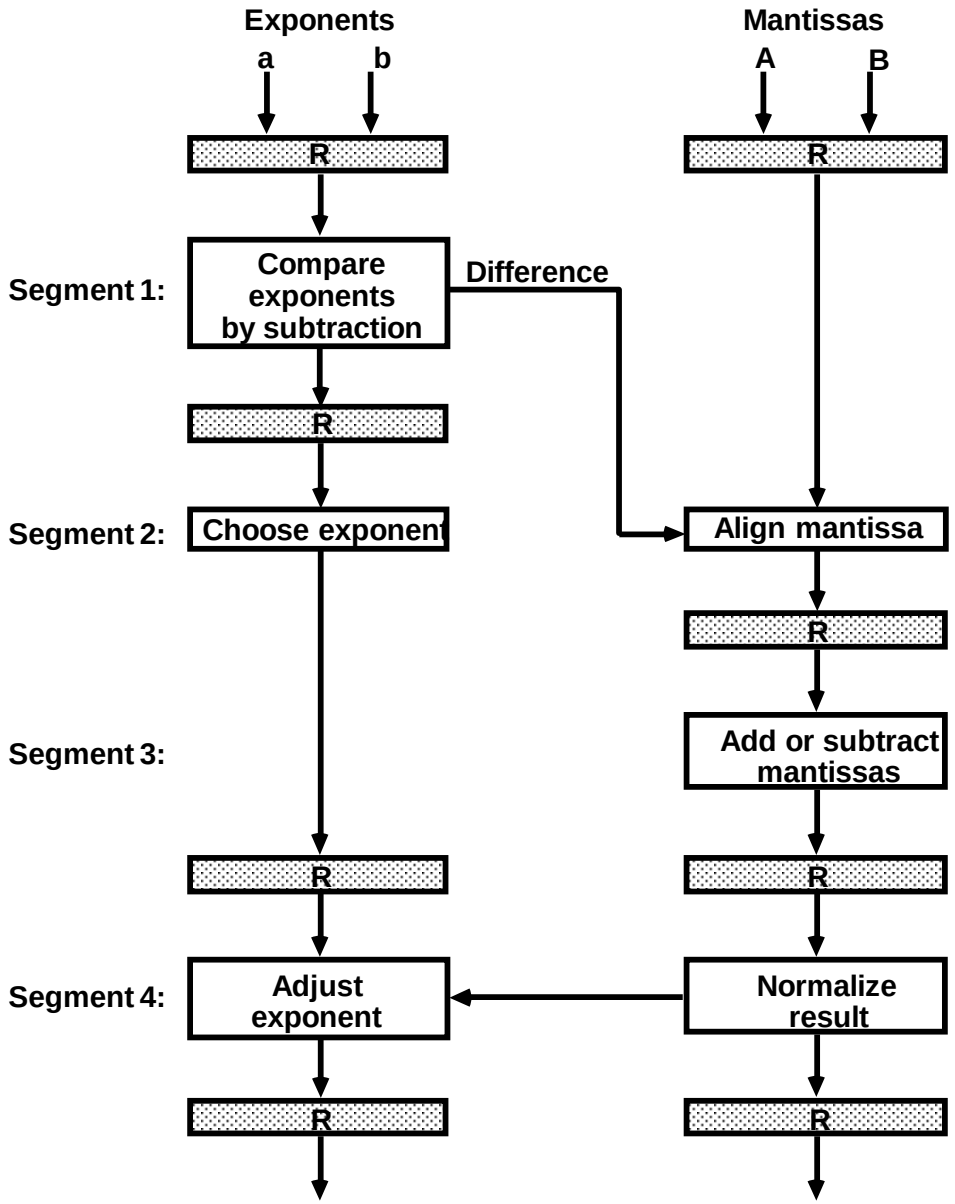
2. The larger exponent is chosen as the exponent of the result.

3. The next segment shifts the mantissa of Y to the right to obtain $=0.0820x10^3$

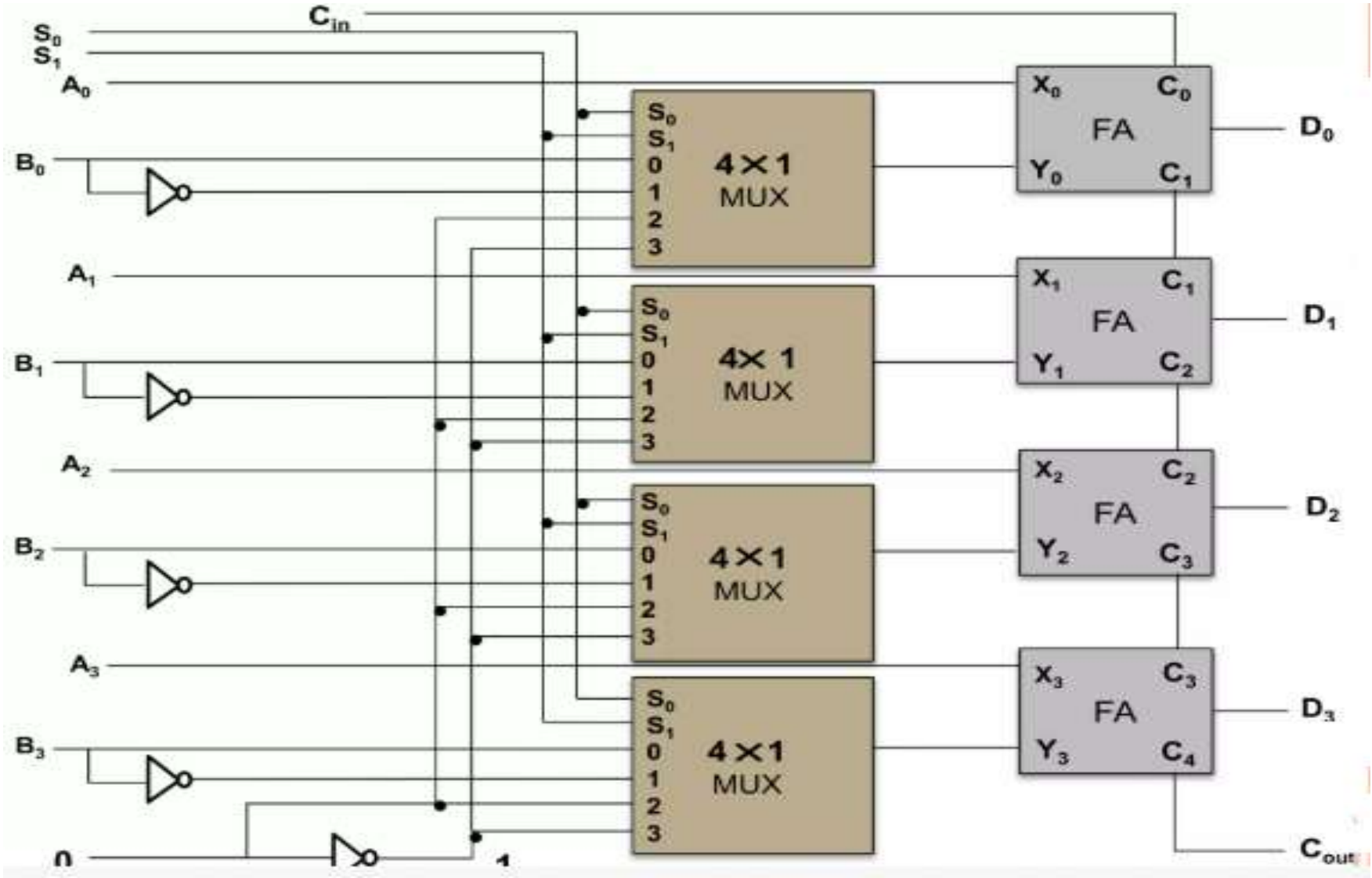
4. This aligns the two mantissas under the Same exponent.

5. Addition of two mantissas in segment 3 produces the sum. $Z = 1.0324x10^3$;

6.Normalize the result; $Z = 0.10324x10^4$



Arithmetic Circuit



Arithmetic Circuit

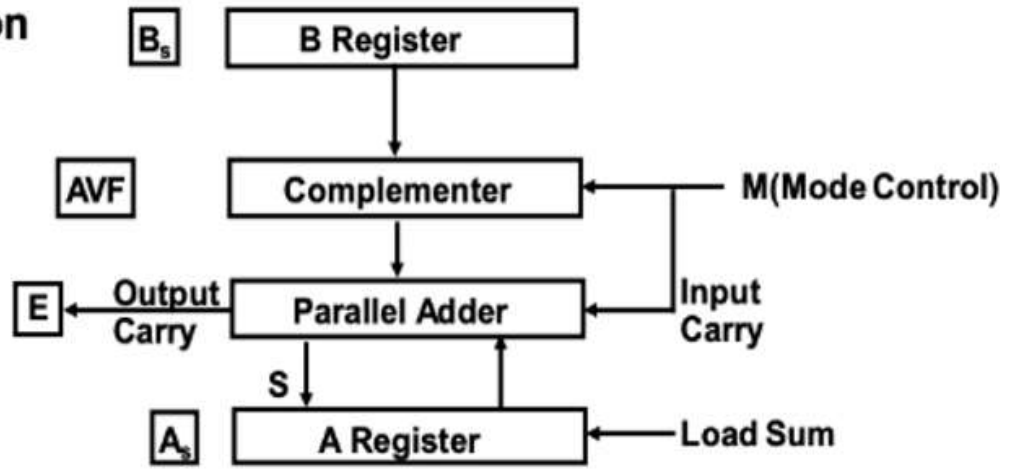
- The basic arithmetic microoperation are (addition, subtraction, increment and decrement) can be performed in one composite arithmetic circuit as under

Select			Input	Output	Micro operation
S_1	S_0	C_{in}	Y	$D = A + Y + C_{in}$	
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with Carry
0	1	0	$\overline{B}$	$D = A + \overline{B}$	Subtract with Borrow
0	1	1	$\overline{B}$	$D = A + \overline{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

Signed Magnitude Addition and Subtraction

Operation	Add Magnitude	Subtract Magnitude		
		When A>B	When A<B	When A=B
(+A) + (+B)	+(A + B)			
(+A) + (- B)		+(A - B)	- (B - A)	+(A - B)
(- A) + (+B)		- (A - B)	+(B - A)	+(A - B)
(- A) + (- B)	- (A + B)			
(+A) - (+B)		+(A - B)	- (B - A)	+(A - B)
(+A) - (- B)	+(A + B)			
(- A) - (+B)	- (A + B)			
(- A) - (- B)		- (A - B)	+(B - A)	+(A - B)

Hardware Implementation



1. As and Bs stores the signed magnitude
2. E is the register of flipflop which hands output carry
3. AVF is the add overflow flip-flop
4. When M=0, the input carry is zero, and the operation undergoing will be of addition
5. When M=1, B will go to complementor resulting B complement; hereby results to the subtraction after the parallel adder comes in operation

Pipelining- CPI

- Pipeline is an implementation technique that exploits parallelism among the instructions in a sequential instruction stream.
- Pipeline allows to overlapping the execution of multiple instructions. A Pipeline is like an assembly line each step or pipeline stage completes a part of an instructions. Each stage of the pipeline will be operating an a separate instruction.
- Instructions enter at one end progress through the stage and exit at the other end.
- If the stages are perfectly balance - assuming ideal conditions - then the time per instruction on the pipeline processor is given by the ratio:

  Time per instruction on unpipelined machine/ Number of Pipeline stages

- If the processor is assumed to take one (long) clock cycle per instruction, then pipelining decrease the clock cycle time. If the processor is assumed to take multiple CPI, then pipelining will aid to reduce the CPI.

Instruction Cycle

- Each step of instruction cycle can be decomposed into micro-operation.
- Micro operation can be defined as the atomic or functional operation of the CPU performed during one clock period. Hence a set of micro-operation occurring at one time is known as micro-operation.
- The processing performed by a CPU to execute a single instruction is now as instruction cycle
- Normally, an instruction cycle includes the following sub cycles:

    1) Fetch the instruction from memory
    2) Decode the instruction and calculate the effective address
    3) Fetch the operands from memory
    4) Execute the instruction
    5) Store the result in the proper place
    6) Recognize the pending interrupt

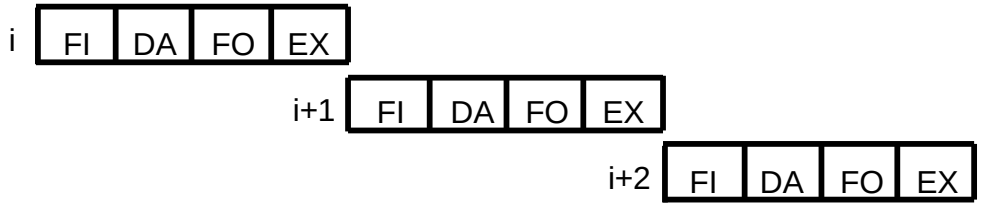
Four-segment Instruction Pipeline

- Four-segment CPU pipeline :
  - » FI : Instruction Fetch
  - » DA : Decode Instruction & calculate effective address
  - » FO : Operand Fetch
  - » EX : Execution

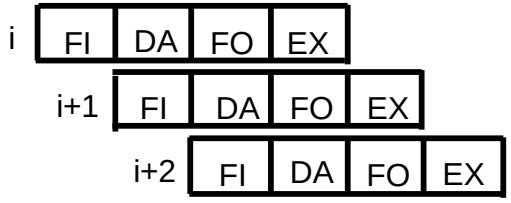
INSTRUCTION PIPELINE

Execution of Three Instructions in a 4-Stage Pipeline

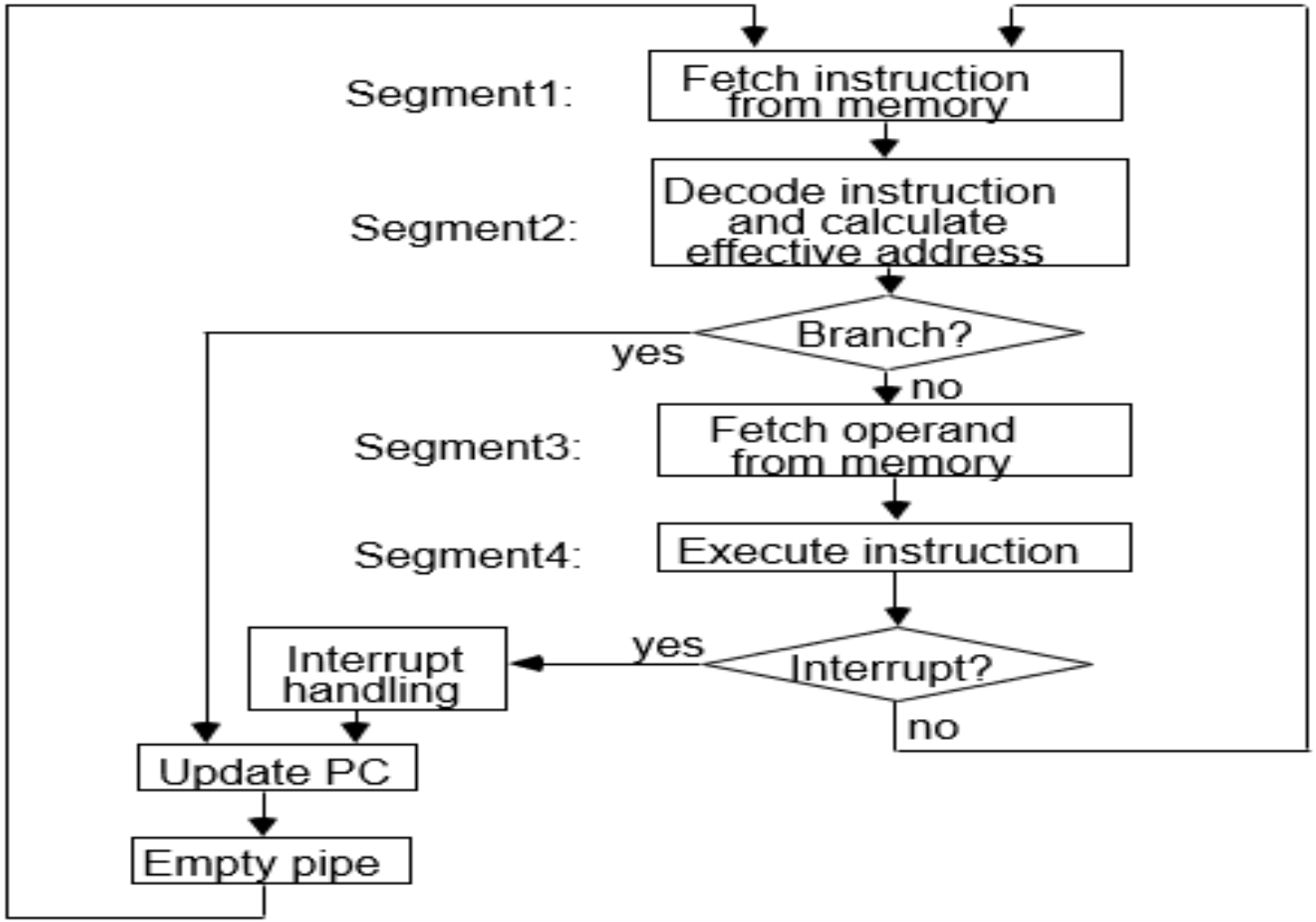
Conventional



Pipelined



INSTRUCTION EXECUTION IN A 4-STAGE PIPELINE



A Simple implementation of a RISC instruction set

- Instruction set of implementation in RISC takes at most 5 cycles without pipelining. The 5 clock cycles are:

1. **Instruction Fetch (IF) Cycle:**
    - Send the content of program count (PC) to memory and fetch the current instruction from memory to update the PC.
2. **Instruction Decode / Register Fetch Cycle (ID):**
    - Decode the instruction and access the register file. Decoding is done in parallel with reading registers, which is possible because the register specifies are at a fixed location in a RISC architecture. This technique is known as fixed-field decoding.

A Simple implementation of a RISC instruction set

3. Execution / Effective address Cycle (EXE)

- The ALU operates on the operands prepared in the previous cycle and performs one of the following function defending on the instruction type.

* **Register- Register ALU instruction:** ALU performs the operation specified in the instruction using the values read from the register file.

- **Register- Immediate ALU instruction:** ALU performs the operation specified in the instruction using the first value read from the register file and that sign extended immediate.

* **Register- Immediate ALU instruction:** Determine whether the condition is true

A Simple implementation of a RISC instruction set

4. Memory Access (MEM):

- For a load instruction, using effective address the memory is read. For a store instruction memory writes the data from the 2nd register read using effective address.

5. Write Back (WB) Cycle

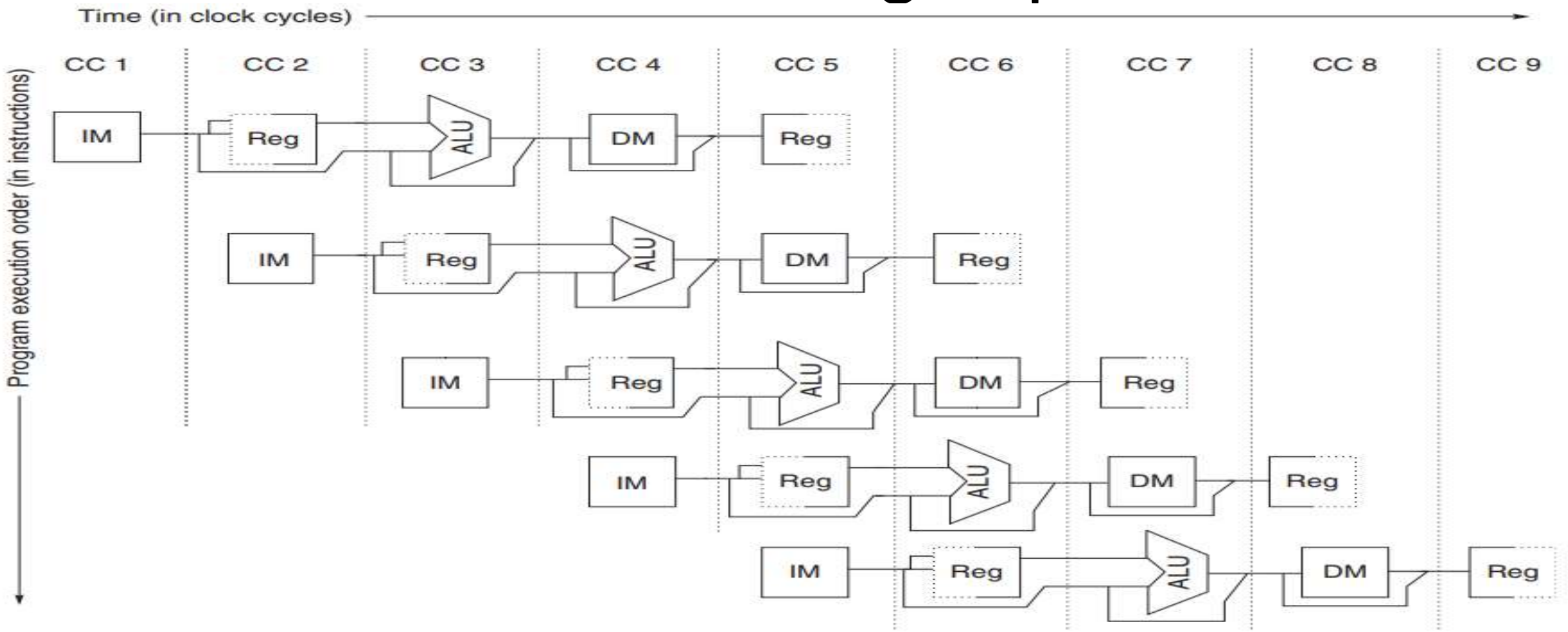
- Write the result in to the register file, whether it comes from memory system (for a LOAD instruction) or from the ALU (for an ALU instruction)

Five stage Pipeline for a RISC processor

	Clock number								
Instruction number	1	2	3	4	5	6	7	8	9
Instruction *i*	IF	ID	EX	MEM	WB				
Instruction *i* + 1		IF	ID	EX	MEM	WB			
Instruction *i* + 2			IF	ID	EX	MEM	WB		
Instruction *i* + 3				IF	ID	EX	MEM	WB	
Instruction *i* + 4					IF	ID	EX	MEM	WB

Simple RISC pipeline. On each clock cycle, another instruction is fetched and begins its five-cycle execution. If an instruction is started every clock cycle, the performance will be up to five times that of a processor that is not pipelined. The names for the stages in the pipeline are the same as those used for the cycles in the unpipelined implementation: IF = instruction fetch, ID = instruction decode, EX = execution, MEM = memory access, and WB = write-back.

Functional Units of a 5 Stage Pipeline



The pipeline can be thought of as a series of data paths shifted in time. This shows the overlap among the parts of the data path, with clock cycle 5 (CC 5) showing the steady-state (a condition where a process, parameter, or variable does not change over time) situation. Because the register file is used as a source in the ID stage and as a destination in the WB stage, it appears twice. We show that it is read in one part of the stage and written in another by using a solid line, on the right or left, respectively, and a dashed line on the other side. The abbreviation IM is used for instruction memory, DM for data memory, and CC for clock cycle.

Basic Performance issues in Pipelining

- Pipelining increases the CPU instruction throughput but, it does not reduce the execution time of an individual instruction.
- In fact, the pipelining increases the execution time of each instruction due to overhead in the control of the pipeline. Pipeline overhead arises from the combination of register delays and clock skew.
- Imbalance among the pipe stages reduces the performance since the clock can run no faster than the time needed for the slowest pipeline stage.

Pipeline Hazards

Pipeline Conflicts : there are three major difficulties that cause the instruction pipeline to deviate from its normal operation

1) **Data dependency (data hazard)**

» Data hazards arise when an instruction depends on the results of a previous instruction in a way that is exposed by the overlapping of instructions in the pipeline.

2) **Resource conflicts (Structural hazard)**

» caused by access to memory by two segments at the same time.

» Can be avoided by using separate instruction stream and data memories.

3) **Branch difficulties (Control hazard)**

» control hazards arise from the pipelining of branches and other instructions (interrupt etc.) that change the PC.

Data Hazard

- Assume instruction i occurs in a program order before instruction j and both instruction use register, then there are three different types of hazards that can occur between i and j
  1. **Read After Write (RAW) hazard**
  2. **Write After Read (WAR) hazard**
  3. **Write After Write (WAW) hazard**

Data Hazard

1. **Read After Write (RAW) hazard**
   - Here j tries to read a source before i writes it, so j incorrectly gets the old value. This hazard is the most common type and corresponds to a true data dependence. Program order must be preserved to ensure that j receives the value from i.
   - Or, a RAW hazard between the two instructions i and j occur when j attempts to read some data object that has been modified by I

$$I_1:\ R_2 \leftarrow R_2 + R_3$$
$$I_2:\ R_5 \leftarrow R_2 + R_4$$

Data Hazard

2. Write After Read (WAR) hazard

- j tries to write a destination before it is read by i, so i incorrectly gets the new value.
- This hazard arises from an anti-dependence (or name dependence).

$$R_1 \leftarrow R_2 * R_3$$

$$R_2 \leftarrow R_4 + R_5$$

Data Hazard

3. Write After Write (WAW) hazard

- Output dependency
- j tries to write an operand before it is written by i.

$$R_3 \leftarrow R_1 * R_2$$

$$R_3 \leftarrow R_4 + R_5$$

Data Hazard

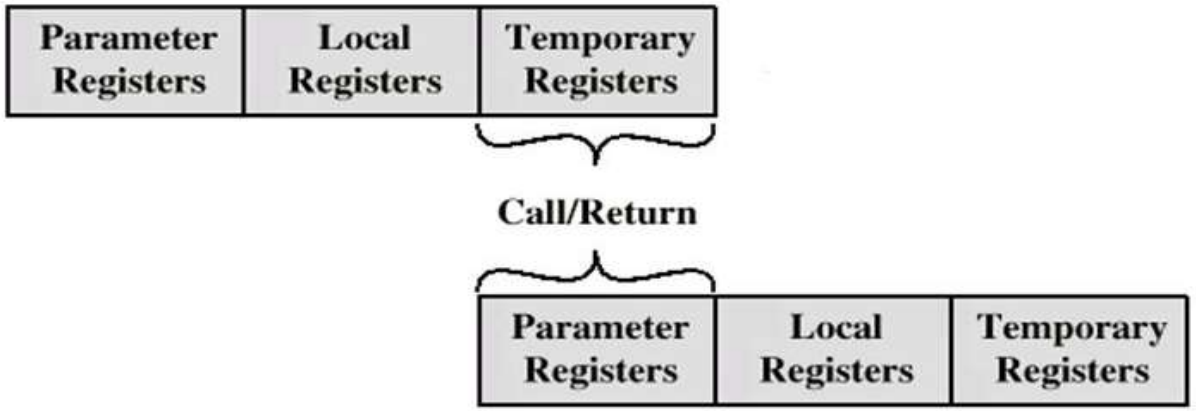
- Data hazard can be dealt with either hardware techniques or software technique
- **Hardware Technique**
  - **Forwarding (bypassing, short-circuiting)** - Accomplished by a data path that routes a value from a source (usually an ALU) to a user, bypassing a designated register. This allows the value to be produced to be used at an earlier stage in the pipeline than would otherwise be possible
  - **Interlock -** Hardware detects the data dependencies and delays the scheduling of the dependent instruction by stalling enough clock cycles.

Data Hazard

- **Software Technique**
  - The compiler is designed to detect a data conflict and reorder instructions
  - As necessary to delay the loading of the conflicting data by inserting no-operation instructions.
  - Instruction Scheduling(compiler) for delayed load; hereby this method is called DELAY LOAD

Data Hazard – Register Window

- A large number of register is usually very helpful
- Contents of all the registers must be saved at every procedure call, however more registers means more delay
- A solution to this problem is to divide the register file into a set of fixed-size window
  i. Each register window is assigned to a procedure
  ii. Windows for adjacent procedures are overlapped to allow parameter passing

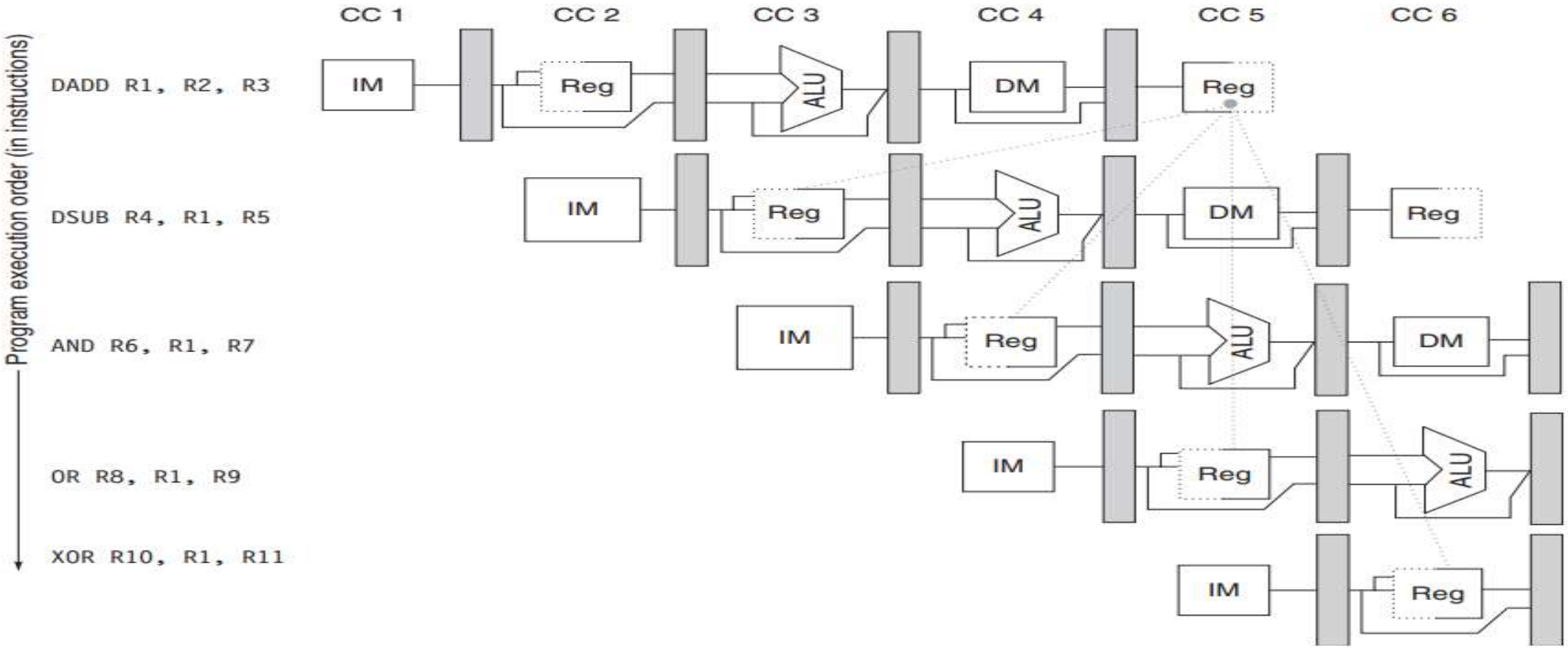


Data Hazard – Register Renaming

- In computer architecture, register renaming is a technique that abstracts logical registers from physical registers
- Register renaming is done in pipelined processors. Register renaming is a technique that removes false data dependencies, such as write after read (WAR) and write after write (WAW) that occur in straight line code between registers and operands of subsequent instructions.

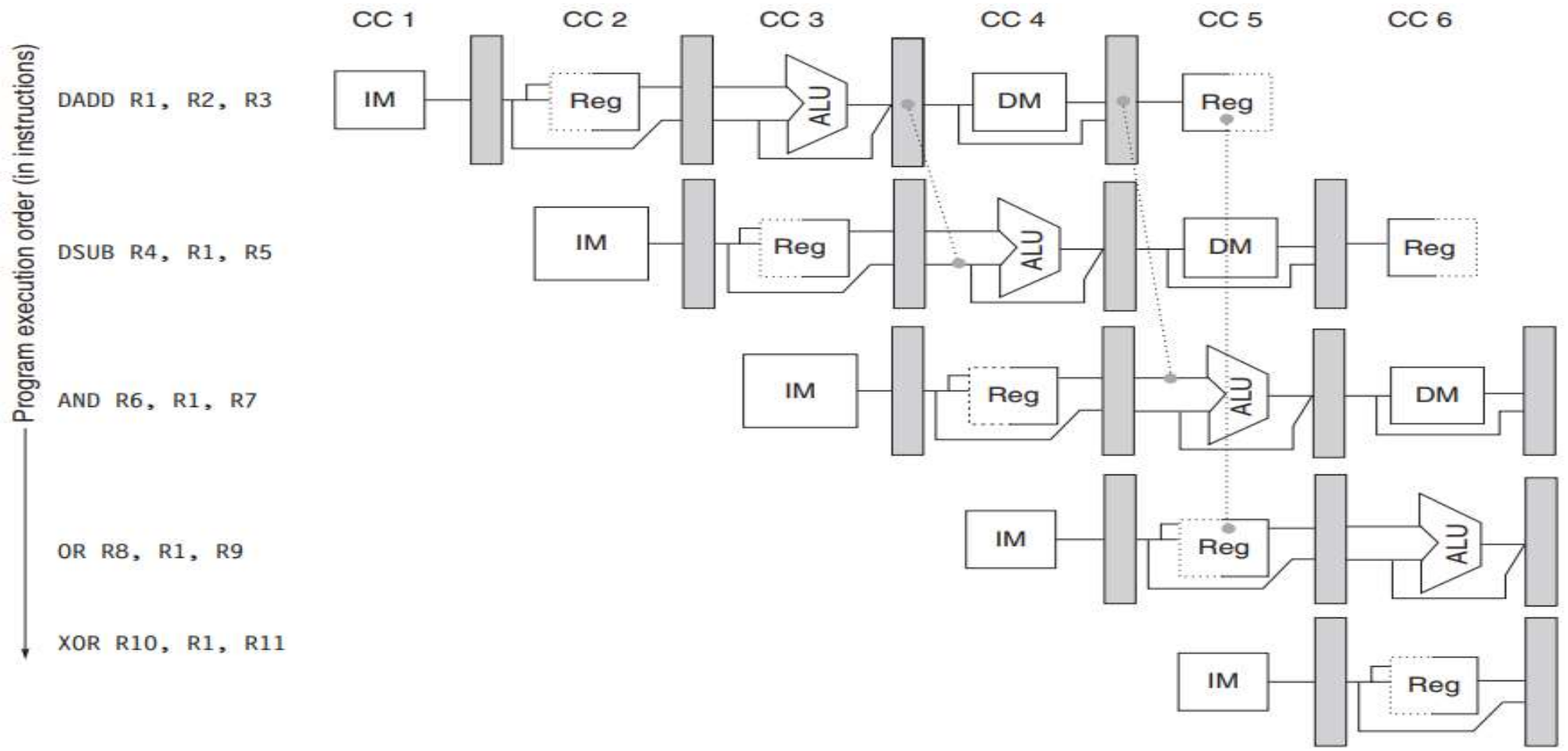
RAW Hazards

- Here the use of the result of the add instruction in the next three instructions cause a hazard, because the register is not written until after those instruction read it



Minimizing data hazards stalls by forwarding

- A set of instructions that depends on the DADD result uses forwarding paths to avoid the data hazard



Data hazards requiring stalls

- Unfortunately not all potential data hazards can be handled by bypassing. Let us consider the following sequence of instructions:

LD	R1,0(R2)	IF	ID	EX	MEM	WB				
DSUB	R4,R1,R5		IF	ID	EX	MEM	WB			
AND	R6,R1,R7			IF	ID	EX	MEM	WB		
OR	R8,R1,R9				IF	ID	EX	MEM	WB	

LD	R1,0(R2)	IF	ID	EX	MEM	WB				
DSUB	R4,R1,R5		IF	ID	stall	EX	MEM	WB		
AND	R6,R1,R7			IF	stall	ID	EX	MEM	WB	
OR	R8,R1,R9				stall	IF	ID	EX	MEM	WB

In the top half, we can see why a stall is needed: The MEM cycle of the load produces a value that is needed in the EX cycle of the DSUB, which occurs at the same time. This problem is solved by inserting a stall, as shown in the bottom half.

INSTRUCTION SCHEDULING

```
a = b + c;
d = e - f;
```

Unscheduled code:

```
    LW    Rb, b
    LW    Rc, c
→   ADD   Ra, Rb, Rc
→   SW    a, Ra
    LW    Re, e
→   LW    Rf, f
→   SUB   Rd, Re, Rf
    SW    d, Rd
```

Scheduled Code:

```
    LW    Rb, b
    LW    Rc, c
    LW    Re, e
    ADD   Ra, Rb, Rc
    LW    Rf, f
    SW    a, Ra
→   SUB   Rd, Re, Rf
    SW    d, Rd
```

Delayed Load

A load requiring that the following instruction not use its result

RISC PIPELINE

RISC

- Machine with a very fast clock cycle that executes at the rate of one instruction per cycle
  <- Simple Instruction Set
    Fixed Length Instruction Format
    Register-to-Register Operations

Instruction Cycles of Three-Stage Instruction Pipeline

Data Manipulation Instructions

I: Instruction Fetch
A: Decode, Read Registers, ALU Operations
E: Write a Register

Load and Store Instructions

I: Instruction Fetch
A: Decode, Evaluate Effective Address
E: Register-to-Memory or Memory-to-Register

Program Control Instructions

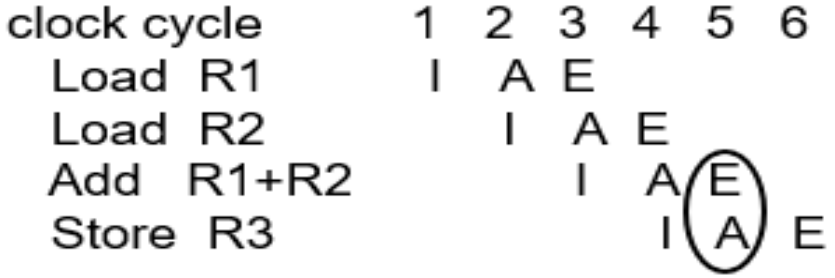
I: Instruction Fetch
A: Decode, Evaluate Branch Address
E: Write Register(PC)

DELAYED LOAD

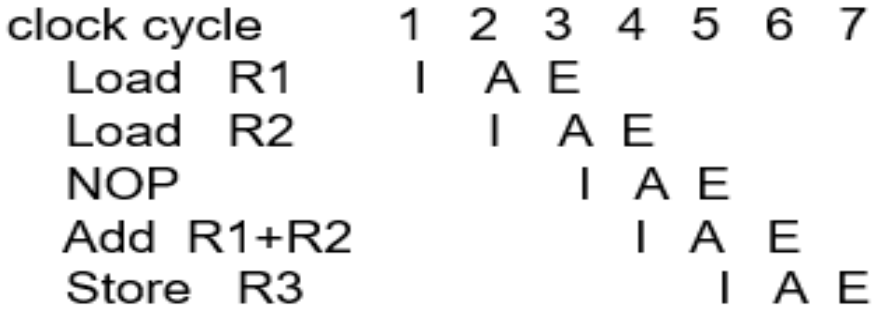
LOAD: R1 ← M[address 1]
LOAD: R2 ← M[address 2]
ADD: R3 ← R1 + R2
STORE: M[address 3] ← R3

Three-segment pipeline timing

Pipeline timing with data conflict



Pipeline timing with delayed load

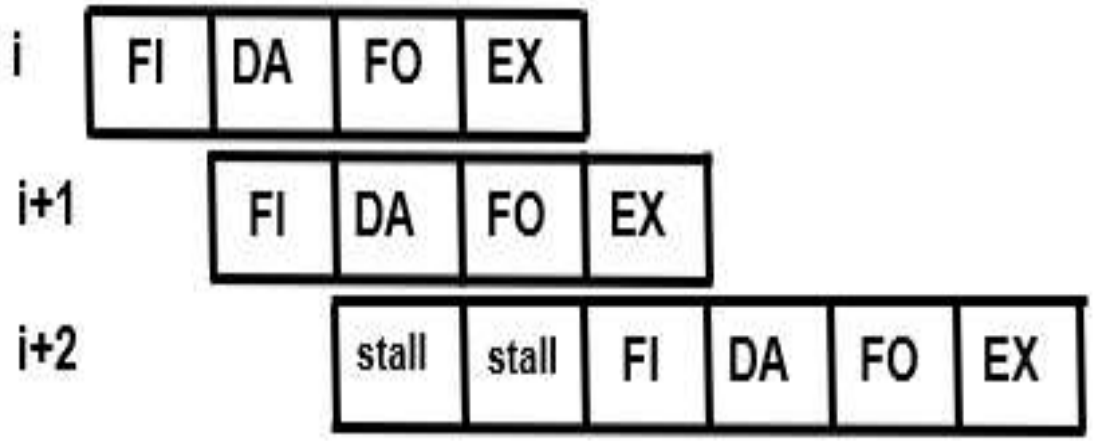


The data dependency is taken care by the compiler rather than the hardware

STRUCTURAL HAZARD

Occur when some resource has not been duplicated enough to allow all combinations of instructions in the pipeline to execute

Example: With one memory, a data and an instruction fetch cannot be initiated in the same clock



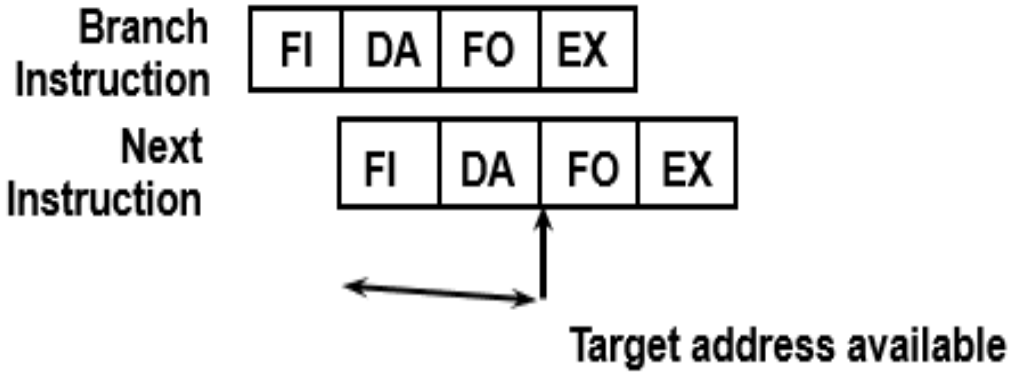
The Pipeline is stalled for resource conflict

STRUCTURAL HAZARD

- It is understood that the “stalls” hamper the performance.
- Hereby other alternatives to structural hazards can be
  - Resource duplication
  - Resource pipeline
  - Resource renaming
  - Instruction ordering

CONTROL HAZARD

- Branch Instructions
  - Branch target address is not known until the branch instruction is decoded.



  - stalls = waste of cycle time

Dealing with CONTROL HAZARD

- **Prefetch Target Instruction**
  - Fetch instructions in both streams, instruction to be executed if branch not taken and the instruction if branch taken
  - Both are saved until branch is executed. Then, select the right instruction stream and discard the wrong stream
- **Branch Target Buffer(BTB; Associative Memory)**
  - Present in the fetch segment of the pipeline. It has entry of the Address of previously executed branches i.e. their target instruction and the next few instructions.
  - When fetching an instruction, search BTB.
    - If found, fetch the instruction stream in BTB;
    - If not, new stream is fetched and update BTB

Dealing with CONTROL HAZARD

- **Loop Buffer (High Speed Register file)**
  - A variation of BTB. A register file maintained by the instruction fetch segment of the pipeline.
  - Register file stores the entire loop that allows to execute a loop without accessing memory
- **Branch Prediction**
  - Uses additional logic to guess the outcome of the branch condition before it is executed.
  - The instruction is fetched based on the guess. Correct guess eliminates the branch penalty
- **Delayed Branch**
  - Compiler detects the branch and rearranges the instruction sequence by inserting useful instructions that keep the pipeline busy in the presence of a branch instruction

Dynamic Instruction Scheduling

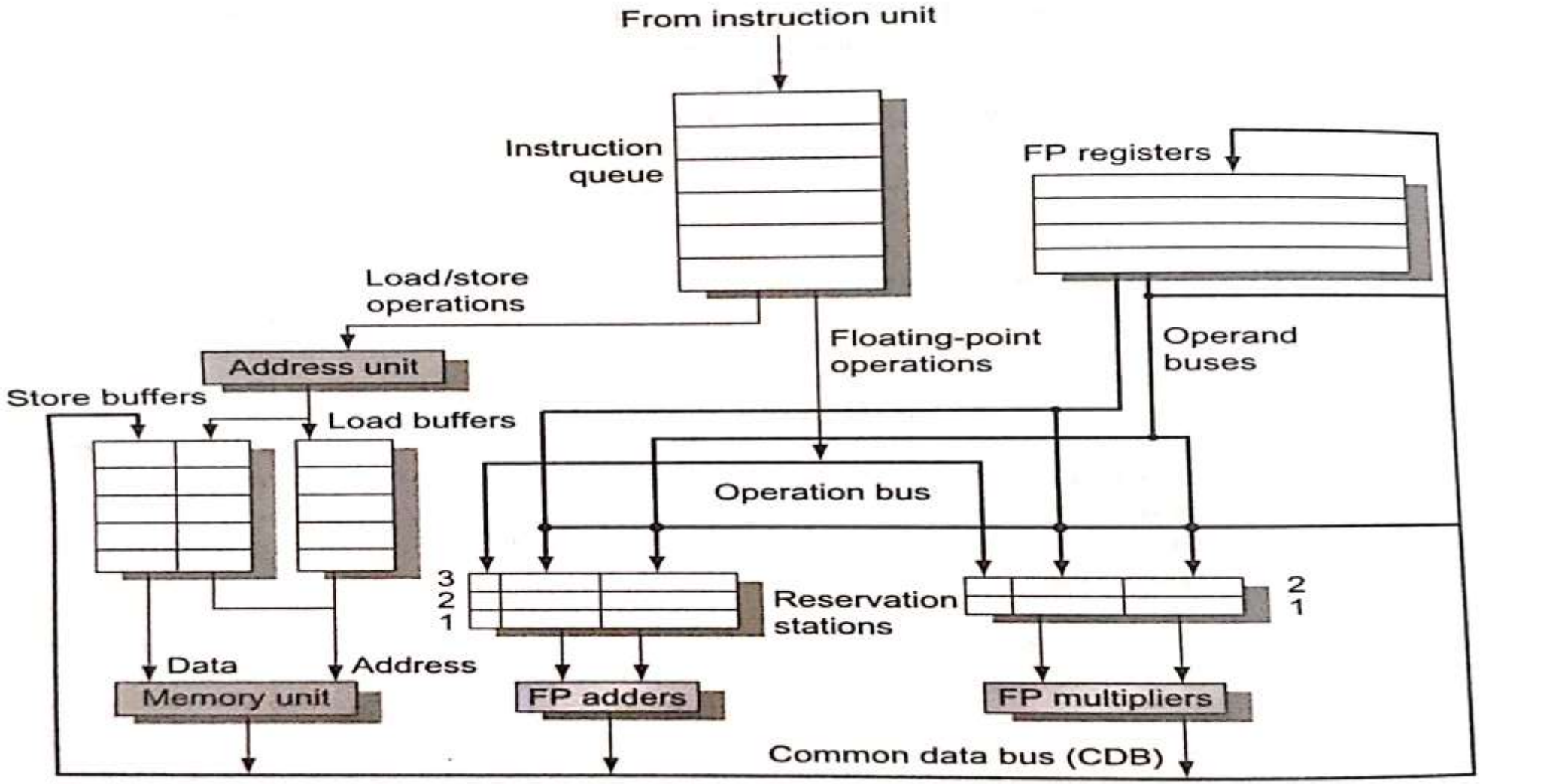
- It is a technique for improving pipeline processor performance
- It is a hardware based approach whereas static scheduling is compiler based approach
- In order to facilitate the compiler scheduling, the hardware needs to communicate with the compiler OR the architectural aspects of the hardware need to be communicated with the complier. Hence the architectural knowledge is a most at the compiler level
- Since compiler level optimization is not always possible, we are now going to explore another area called dynamic scheduling to explore ILP.
- So in dynamic scheduling we will see how we can equip the computer hardware to reduce stalls and improve ILP without much help and support from the compiler

Dynamic Instruction Scheduling – Tomasulo's Approach

- Used in IBM 360 machines
- It determines which instruction have inputs ready so that they can go in the next cycle and which instructions still have to wait for their inputs to be produced
- It includes a form of register renaming, and it is similar to what we use today as for as out of order execution is concerned
- It is very similar to what we use today

Tomasulo	Today
Floating point instructions	Today we do it for all instructions
Fewer instructions in "window"	Hundreds of instructions
Exception handling	Today we have explicit support for exceptional handling

Dynamic Instruction Scheduling – Tomasulo's Approach



Vector Database

- A vector database stores, manages and indexes high-dimensional vector data.
- Data points are stored as arrays of numbers called “vectors,” which are clustered based on similarity.
- This design enables low-latency queries, making it ideal for AI applications.

Vector Instruction Types

There are six types of vector instructions for register based pipelined vector machines. So whatever vectors we have are stored on registers; and the vector machines are pipelined

1. Vector-vector instructions
2. Vector-scalar instructions
3. Vector memory instructions
4. Vector reduction instructions
5. Gather and scatter instructions
6. Masking instructions

Vector Performance Modeling

The performance of a pipelined vector processor is measured on the basis of the following parameters:

- Speed up
- Utilization efficiency
- Throughput
- Performance to cost ratio

Vector Performance Modeling –speed up

Vector processor speed can be increased by reducing processor overhead, improving instructions, and choosing the right algorithm

- **Reduce processor overhead**
  - Improve the pipeline: Integrate the same type of scalar instruction into the pipeline
  - Reduce memory access: Improve vector instructions by reducing memory access
- **Improve instructions**
  - Add vector registers: Add vector registers that can store vectors of a fixed size
  - Add vector functional units: Add vector functional units that can operate on the vector registers
  - Add a vector load/store unit: Add a vector load/store unit to interface the vector registers to the memory
- **Choose the right algorithm**
  - Pick an algorithm that works faster for the use case
  - Choose an algorithm that can be easily integrated with processors
  - Use a vectorizing compiler to regenerate parallelism

Vector Performance Modeling –speed up

- The speed-up factor is given as the ratio of the execution time for one processor to that of the ‘P’ processors. It can also be said as the ratio of the speed of ‘P’ processors executing simultaneously to that of the single processor.

$$S_p = \frac{\text{Execution time using one processor}}{\text{Execution time using P processors}}$$

- Also when comparing the times required to execute the problem in single and parallel processors, the time to be considered on the sequential processors must be the best algorithm time required. Hence we can say the speed-up ratio can be given as

$$S_p = \frac{\text{Execution time for the best serial algorithm on one}}{\text{Execution time for parallel algorithm on P processors}}$$

Vector Performance Modeling - Utilization efficiency

- Vector processors are central processing units (CPUs) that are designed to efficiently process large amounts of data.
- They are more efficient than scalar processors because they can perform multiple operations on data simultaneously.

Vector Performance Modeling - Throughput

- The throughput of a vector processor depends on the memory system used and the number of pipes in the ALU.

Vector Performance Modeling – Cost-performance ratio

- **Cost**
  - Vector processors can be more expensive than conventional CPUs, but they can also be faster. The cost-to-performance ratio of vector processors depends on the type of application and the software used.
- **Performance**
  - Vector processors can be faster because they initiate a single access for the entire vector, rather than for each word
- **Software**
  - Vector processing requires specialized, hardware-optimized software.
  - This can be a constraint for some applications, especially those that need to work with a variety of software environments.

Vectorization

- Vectorization is the process of converting data into numerical vectors that represent essential features. This transformation is especially useful in fields like natural language processing (NLP) and computer vision, where unstructured data needs to be represented in a form that algorithms can understand.
- Vectorization is a special case of Single Instructions Multiple Data (SIMD) to denote a single instruction stream capable of operating on multiple data elements in parallel

SIMD Interconnection Networks

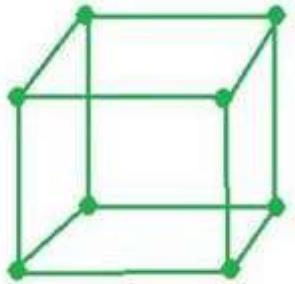
- Interconnection Networks, also called multi-stage interconnection network are high speed computer network
- They are connection between odes where each node can be a single processor or a group of processor or memory modules
- These connections carry or transport data from one processor to another or from the processor to the memory so that the task is broken down and computed parallel

1. Static versus Dynamic Network
2. Cube Interconnection Network
3. Hypercube Interconnection Network
4. Mesh Connected Illiac Network

Static versus Dynamic Network

- Static network
  - They are composed of point to point connections
  - The links between the two processors are passive. They do not change during program execution
  - They are classified as 1D, 2D, 3D or as a hypercube
  - Examples: linear array, ring, star, mesh etc.
- Dynamic network
  - They are composed of switched channels
  - The link between the processors are active or dynamic. Switches are dynamic
  - Examples: digital bus, switches, crossbar etc.

Cube Interconnection Network

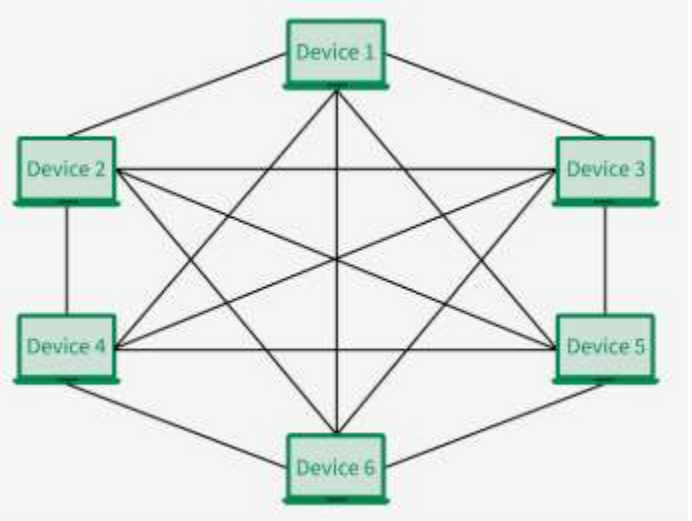


Three Cube Network

- A cube interconnection network is a network topology that uses a cube to connect nodes. It can be used in parallel and distributed supercomputer systems.
- How it works?
  - Each node is assigned a binary string with n bits
  - Two nodes can communicate if their binary strings differ by exactly one bit
  - The network can be partitioned into subnetworks
- Examples of cube interconnection networks
  - Hypercube: An extension of the cube network, where each node is assigned a binary string
- Applications
  - Staran, Pasm, Ultracomputer, BBN Butterfly, IBM RP3, and Data-flow machines.

Mesh Connected Illiac Network

- A mesh connected Illiac network is a type of computer network that uses a mesh topology to connect nodes, or devices, to each other. The Illiac-IV type of mesh-connected multiprocessor uses a wraparound mesh topology



Mesh Connected Illiac Network

- Benefits of mesh networks
  - Redundancy: If one connection fails, data can be routed through another connection.
  - Scalability: It's easy to add nodes to a mesh network.
  - Reliability: If one node fails, its neighbors can find another route.
  - Data handling: Mesh networks can handle large amounts of data traffic

Arrays

- Array is a collection of elements of similar data types. So if we want to store multiple values under a single variable name, we use arrays.
- Array variables can be initialized by two types namely dynamic or run time initialization and static or compile time initialization
- In arrays we can access the data at any position during the constant time i.e. O(1)
- All the elements inside an array gets memory allocation and we are accessing these elements using index numbers. So we can say array is a internal pointer which holds the base address of the memory block.

  int arr[5] = {10, 20, 30, 40, 50}
- Array is a contiguous memory location. The only disadvantage of using array is that we need to give the size of the array at the starting time.

Array Processors

- An array processor is a processor that performs the computations on large arrays of data.
- There are two different types of array processor:
  - Attached Array Processor
  - SIMD Array Processor

Attached Array Processor

- It is designed as a peripheral for a conventional host computer. Its purpose is to enhance the performance of the computer
- It achieves high performance by means of parallel processing with multiple functional units.

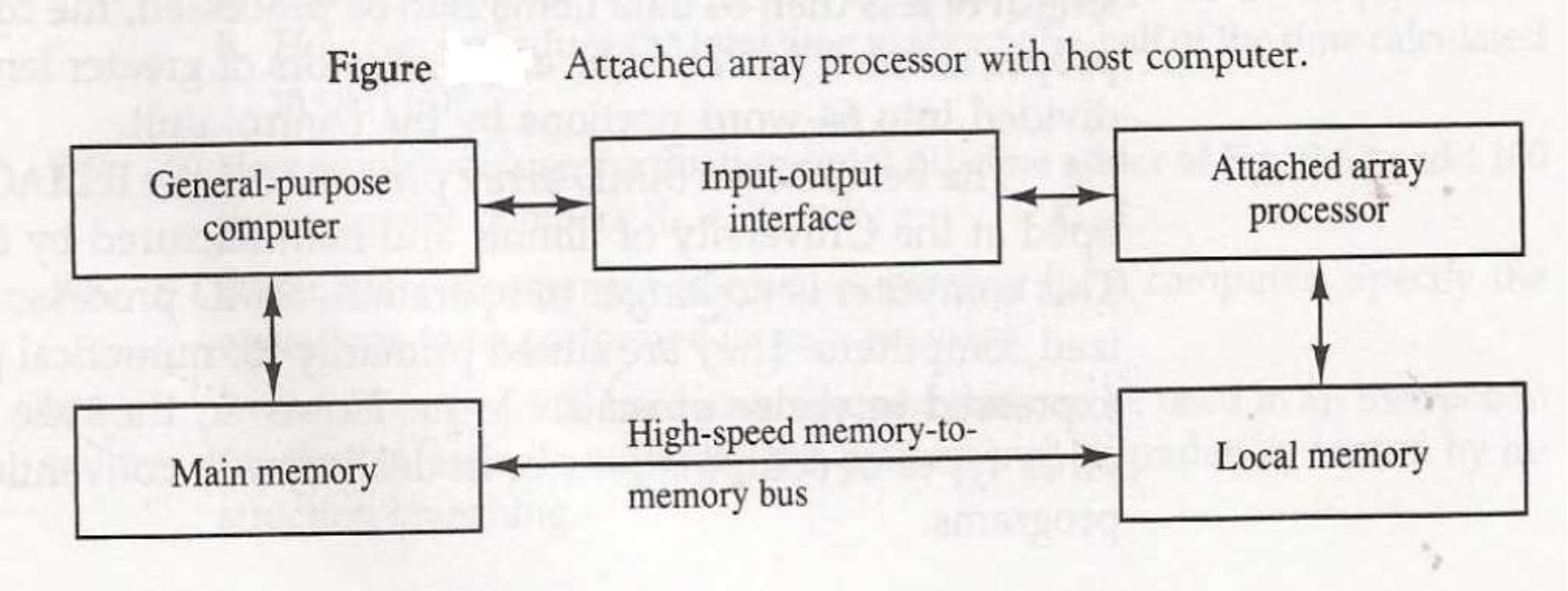


Figure Attached array processor with host computer.

SIMD Array Processor

- It is processor which consists of multiple processing unit operating in parallel.
- The processing units are synchronized to perform the same task under control of common control unit.
- Each processor elements(PE) includes an ALU , a floating point arithmetic unit and working register.

SIMD Array Processor

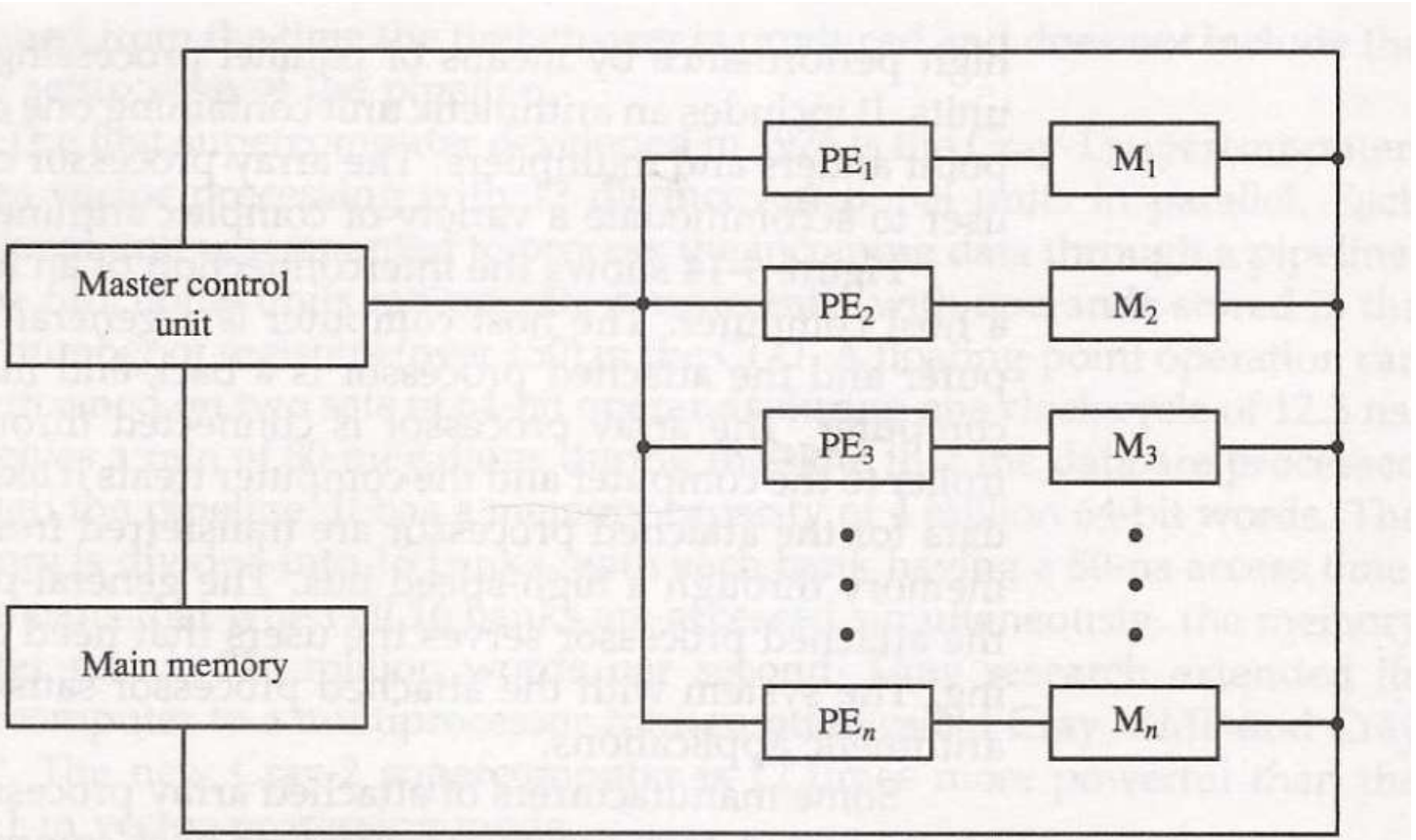


Figure SIMD array processor organization.

SIMD Array Processor

- Here we don’t have a single adder, but here we have a multiple adders
- And when we have a processing engine or a processing elements, it needs a local memory.
- It is now the job of the control unit to first allocate data across these memory elements in respective processing elements and then give an instruction to these processing elements to compute on the local data and produce the answer and store it in its local memory.

SIMD Parallel Algorithms - Matrix Multiplication

There are two methods of matrix multiplication for a mesh organizations

1. Cannon's Algorithm
2. 2D-Pipeline-Systolic Array

Cannon's Algorithms - Matrix multiplication

- Algorithms works in $\sqrt{P}$ steps, where 'P' is the number of processors
- Step1
  - circular shift i^{th} row of matrix A's blocks 'i' positions to the left
  - circular shift j^{th} column of matrix B's blocks 'j' positions up
- Each processor P_{ij} locally updates matrix block C_{ij} in parallel
- Step2
  - circular shift $each$ row of matrix A's blocks 1 positions to the left
  - circular shift $each$ column of matrix B's blocks 1 positions up
- Each processor P_{ij} multiplies the elements brought to it and adds the results to the accumulating sum

Cannon's Algorithms - Matrix multiplication

- Eg. A= $\begin{bmatrix} 2 & 3 & 4 & 5 \\ 9 & 8 & 7 & 6 \\ 5 & 4 & 2 & 3 \\ 8 & 7 & 3 & 4 \end{bmatrix}$ B= $\begin{bmatrix} 3 & 5 & 7 & 6 \\ 2 & 7 & 6 & 3 \\ 7 & 5 & 3 & 2 \\ 4 & 3 & 2 & 5 \end{bmatrix}$

- Here total number of processes in the above example with 4 rows and 4 columns will be p = 16. Therefore total number of steps/iteration will be $\sqrt{16}$ = 4

Step 1

Find A_1 from A by following process

$$A = \begin{bmatrix} 2 & 3 & 4 & 5 \\ 9 & 8 & 7 & 6 \\ 5 & 4 & 2 & 3 \\ 8 & 7 & 3 & 4 \end{bmatrix} \begin{matrix} \leftarrow 0 \text{ left shift} \\ \leftarrow 1 \text{ left shift} \\ \leftarrow 2 \text{ left shift} \\ \leftarrow 3 \text{ left shift} \end{matrix}$$

$$A_1 = \begin{bmatrix} 2 & 3 & 4 & 5 \\ 8 & 7 & 6 & 9 \\ 2 & 3 & 5 & 4 \\ 4 & 8 & 7 & 3 \end{bmatrix}$$

find B_1 from B.

$$B = \begin{bmatrix} 3 & 5 & 7 & 6 \\ 2 & 7 & 6 & 3 \\ 7 & 5 & 3 & 2 \\ 4 & 3 & 2 & 7 \end{bmatrix}$$

0 upshift, 1 up shift, 2 up shift, 3 up shift.

$$B_1 = \begin{bmatrix} 3 & 7 & 3 & 5 \\ 2 & 5 & 2 & 6 \\ 7 & 3 & 7 & 3 \\ 4 & 5 & 6 & 2 \end{bmatrix}$$

$$C_1[i][j] = A_1[i][j] * B_1[i][j]$$

$$\therefore C_1 = \begin{bmatrix} 6 & 21 & 12 & 25 \\ 16 & 35 & 12 & 54 \\ 14 & 9 & 35 & 12 \\ 16 & 40 & 42 & 6 \end{bmatrix}$$

Step2.

find A_2 from A_1

$$A_1 = \begin{bmatrix} 2 & 3 & 4 & 5 \\ 8 & 7 & 6 & 9 \\ 2 & 3 & 5 & 4 \\ 4 & 8 & 7 & 3 \end{bmatrix} \begin{matrix} \leftarrow 1 \text{ left shift} \\ \leftarrow 1 \text{ left shift} \\ \leftarrow 1 \text{ left shift} \\ \leftarrow 1 \text{ left shift} \end{matrix}$$

$$A_2 = \begin{bmatrix} 3 & 4 & 5 & 2 \\ 7 & 6 & 9 & 8 \\ 3 & 5 & 4 & 2 \\ 8 & 7 & 3 & 4 \end{bmatrix}$$

find B_2 from B_1.

$$B_1 = \begin{bmatrix} 3 & 7 & 3 & 5 \\ 2 & 5 & 2 & 6 \\ 7 & 3 & 7 & 3 \\ 4 & 5 & 6 & 2 \end{bmatrix}$$

↑ 1 up shift ↑ 1 up shift ↑ 1 up shift ↑ 1 up shift

$$B_2 = \begin{bmatrix} 2 & 5 & 2 & 6 \\ 7 & 3 & 7 & 3 \\ 4 & 5 & 6 & 2 \\ 3 & 7 & 3 & 5 \end{bmatrix}$$

$$C_2[i][j] = A_2[i][j] \times B_2[i][j]$$

$$\therefore C_2 = \begin{bmatrix} 6 & 20 & 10 & 12 \\ 49 & 18 & 63 & 24 \\ 12 & 25 & 24 & 4 \\ 24 & 49 & 9 & 20 \end{bmatrix}$$

Step 3.

find A_3 from A_2

$$A_2 = \begin{bmatrix} 3 & 4 & 5 & 2 \\ 7 & 6 & 9 & 8 \\ 3 & 5 & 4 & 2 \\ 8 & 7 & 3 & 4 \end{bmatrix}$$

← 1 left shift
← 1 left shift
← 1 left shift
← 1 left shift

$$A_3 = \begin{bmatrix} 4 & 5 & 2 & 3 \\ 6 & 9 & 8 & 7 \\ 5 & 4 & 2 & 3 \\ 7 & 3 & 4 & 8 \end{bmatrix}$$

find B_3 from B_2.

$$B_2 = \begin{bmatrix} 2 & 5 & 2 & 6 \\ 7 & 3 & 7 & 3 \\ 4 & 5 & 6 & 2 \\ 3 & 7 & 3 & 5 \end{bmatrix}$$

↑ 1 up shift
↑ 1 up shift
↑ 1 up shift
↑ 1 up shift.

$$B_3 = \begin{bmatrix} 7 & 3 & 7 & 3 \\ 4 & 5 & 6 & 2 \\ 3 & 7 & 3 & 5 \\ 2 & 5 & 2 & 6 \end{bmatrix}$$

$$C_3[i][j] = A_3[i][j] * B_3[i][j]$$

$$\therefore C_3 = \begin{bmatrix} 28 & 15 & 14 & 9 \\ 24 & 45 & 48 & 14 \\ 15 & 28 & 6 & 15 \\ 14 & 15 & 8 & 48 \end{bmatrix}$$

Step 4:

Find A_4 from A_3

$$A_3 = \begin{bmatrix} 4 & 5 & 2 & 3 \\ 6 & 9 & 8 & 7 \\ 5 & 4 & 2 & 3 \\ 7 & 3 & 4 & 8 \end{bmatrix} \begin{matrix} \leftarrow 1 \text{ left shift} \\ \leftarrow 1 \text{ left shift} \\ \leftarrow 1 \text{ left shift} \\ \leftarrow 1 \text{ left shift} \end{matrix}$$

$$A_4 = \begin{bmatrix} 5 & 2 & 3 & 4 \\ 9 & 8 & 7 & 6 \\ 4 & 2 & 3 & 5 \\ 3 & 4 & 8 & 7 \end{bmatrix}$$

Find B_4 from B_3

$$B_3 = \begin{bmatrix} 7 & 3 & 7 & 3 \\ 4 & 5 & 6 & 2 \\ 3 & 7 & 3 & 5 \\ 2 & 5 & 2 & 6 \end{bmatrix}$$

$\uparrow^{1}$ up shift, $\uparrow^{1}$ up shift, $\uparrow^{1}$ up shift, $\uparrow^{1}$ up shift

$$B_4 = \begin{bmatrix} 4 & 5 & 6 & 2 \\ 3 & 7 & 3 & 5 \\ 2 & 5 & 2 & 6 \\ 7 & 3 & 7 & 3 \end{bmatrix}$$

$$C_4[i][j] = A_4[i][j] * B_4[i][j]$$

$$\therefore C_4 = \begin{bmatrix} 20 & 10 & 18 & 8 \\ 27 & 56 & 21 & 30 \\ 8 & 10 & 6 & 30 \\ 21 & 12 & 56 & 21 \end{bmatrix}$$

Hence, final Result is $C = C_1 + C_2 + C_3 + C_4$

$$\therefore C = \begin{bmatrix} 60 & 66 & 54 & 54 \\ 116 & 154 & 144 & 122 \\ 49 & 72 & 71 & 61 \\ 75 & 116 & 115 & 95 \end{bmatrix}$$

Cannon’s Algorithms - Matrix multiplication

- The time complexity of Cannon's algorithm for matrix multiplication is O(n^3), which is the same as the time complexity of a standard sequential matrix multiplication algorithm, where "n" represents the dimension of the square matrices involved; however, Cannon's algorithm is designed to be significantly faster on parallel computing systems by distributing the computation across multiple processors.
- **O(n^3) complexity:**
  - Like a standard sequential matrix multiplication, Cannon's algorithm also has a time complexity of O(n^3) as it performs the same number of basic operations.
- **Parallel processing advantage:**
  - The key benefit of Cannon's algorithm is its ability to efficiently distribute the computation across multiple processors in a parallel computing environment, allowing for faster execution on large matrices.
- **Mesh topology:**
  - Cannon's algorithm is particularly well-suited for 2D mesh network topologies, where processors are arranged in a grid-like structure

Systolic Architecture

- Replace single processor with an array of regular processing elements (PE) and carefully orchestrate flow of data between the PEs data for high throughput with less memory access
- Different from pipelining
  - Non linear array structure, multidirectional data flow, each processing elements may have small local instruction and data memory



Systolic Array – Matrix Multiplication

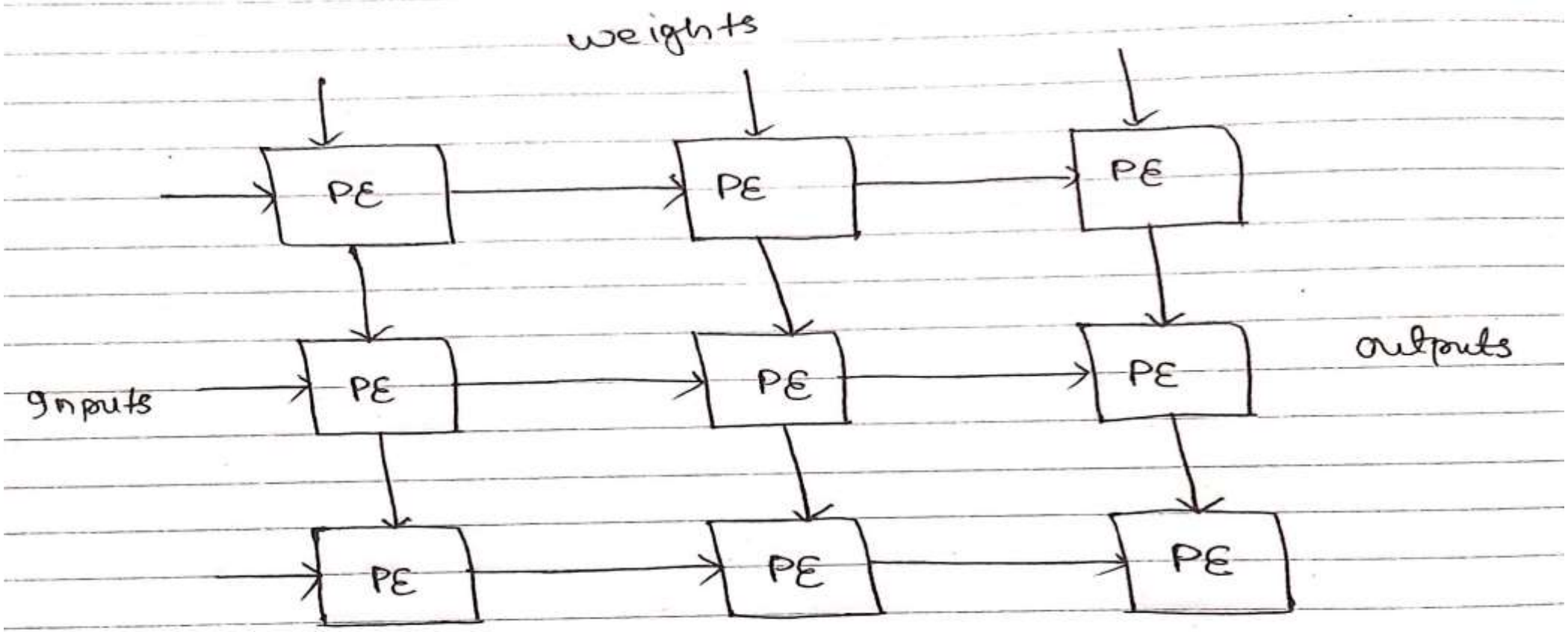
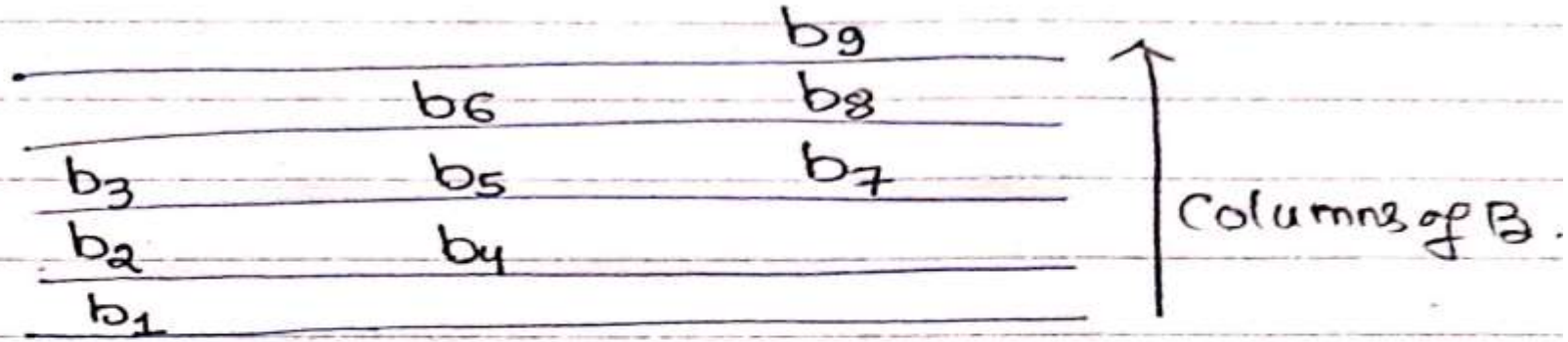
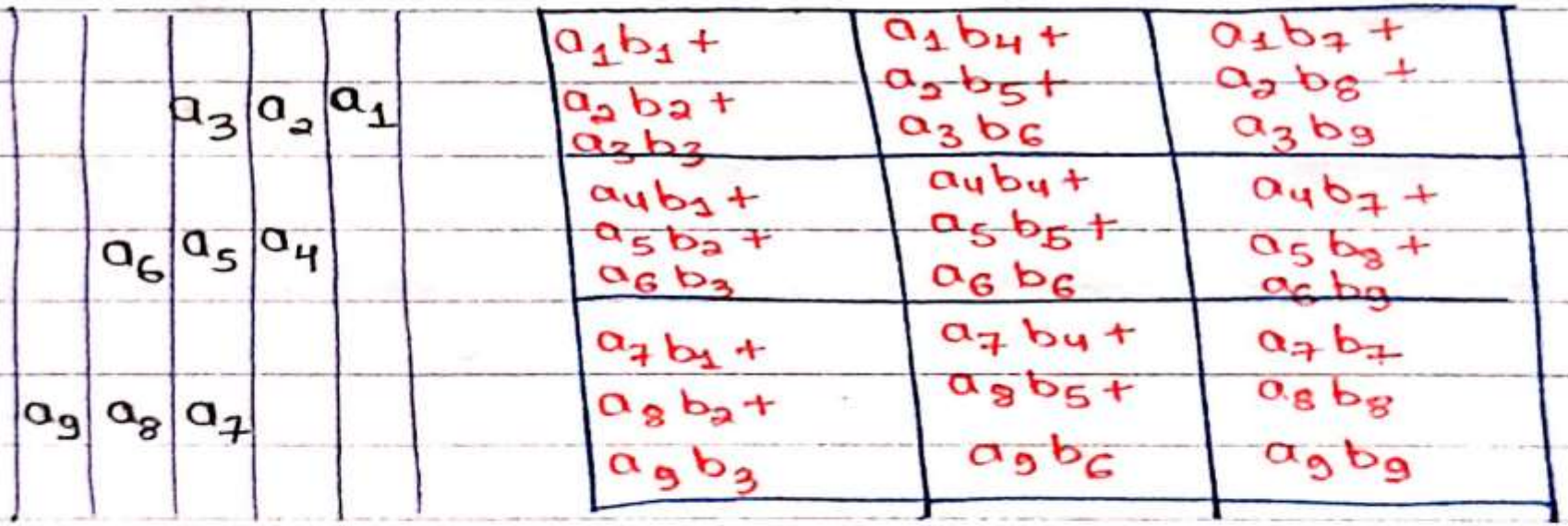


fig. 3x3 multiplication using systolic arrays.

3x3 Systolic Array – Matrix Multiplication





Rows of A.

Alligments in time.

3x3 Systolic Array – Matrix Multiplication

- Processors are arranged in a 2D grid
- We have matrices A and B which are both 3 cross 3 matrices
- We have rows of A in the alignment of time and columns of B
- The elements of B will go from top to bottom and the elements of A will go from left to right
- Each of the squares (called as processing elements) does multiplication operation
- Each processor accumulates one element of the product

3x3 Systolic Array – Matrix Multiplication

- At time, t = 1, only one multiplication is performed i.e. a1*b1. In this cycle other eight processing elements don’t do any computation
- At time, t = 2, a1 moves from left to right whereas b1 goes to one step down.
- Now one more elements form a and b will enter to the systolic area. It means for every cycle, one more element will come and in the existing element if they have not reached to their final processing elements(PE)., they will further move down
- Here every processor doesn’t get the data directly from the memory, rather, it gets the data which was processed in the previous cycle by the PE. It shows that in systolic array, the data movement is not from the memory, the data movement here is local.

3x3 Systolic Array – Matrix Multiplication

- Finally at t = 7, the computation for every processing elements completes
- With the power of parallel processing, dynamic programming and with the help of some geometry, we have dramatically shrink the amount of time steps needed to multiply these matrices down to 14 steps
- In the brute force method, we had to use every matrix element multiple times. Hereby with this idea of **sliding the rows and columns** of the matrices together, we only have to use each matrix element once

Vector Processing

- Vector is known as dynamic array with the ability to resize itself automatically when an element is inserted or deleted with their storage being handled automatically
- There is a class of computational problems that are beyond the capabilities of the conventional computer. These are characterized by the fact that they require vast number of computation and it take a conventional computer days or even weeks to complete.

Vector Processing

- Computers with vector processing are able to handle such instruction and they have application in following fields:
  - Long range weather forecasting
  - Petroleum exploration
  - Seismic data analysis
  - Medical diagnosis
  - Aerodynamics and space simulation
  - Artificial Intelligence and expert system
  - Mapping the human genome
  - Image Processing

Vector Processing

- In computing world, a vector processing is a CPU that implements an instruction set containing instructions that operates on 1D array of data called vectors; compared to scalar processors whose instructions operate on simple data items
- Vector processor have high level operations that work on linear arrays of numbers or vectors
- Scalar quantity 

- Vector quantity 

Vector Operation

- A vector V of length n is represented as row vector by

$$V = [V_1 \quad V_2 \quad V_3 \quad \cdots \quad V_n]$$

- The element V_i of vector V is written as V(I) and the index I refers to a memory address or register where the number is stored.

- Let us consider the program in assembly language that two vectors A and B of length 100 and put the result in vector C.

```
      Initialize I = 0
20    Read A(I)
      Read B(I)
      Store C(I) = A(I) + B(I)
      Increment I = I + 1
      If I ≤ 100 go to 20
      Continue
```

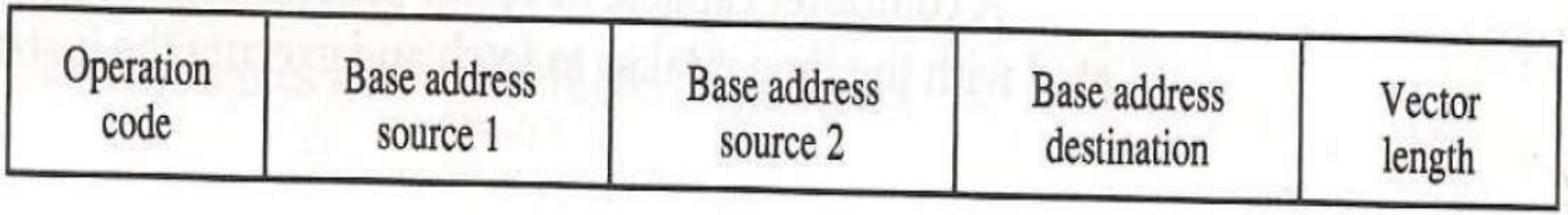
Vector Processing

- Vector processors have the ability to remove the overhead associated with the time it takes to fetch and execute the instructions in the program loop.
- For a single vector instruction, the instruction can be written as:

$$C(1:100) = A(1:100) + B(1:100)$$

Vector Instruction Format

Figure Instruction format for vector processor.



In Conclusion

- Compared to scalar processor which do one instructions at a time and these scalar processors exploits instruction level parallelism (ILP) in the best way. But to exploit thread level parallelism (TLP), we need more processing elements
- *Question* ➔ So apart form multicores, are there designs which helps us to do execution of multiple instructions at a time?
- *Answer* ➔ so vector processors and array processors gives us this features where vector processors use pipelining whereas array processor use parallelism

Task3

1. What is arithmetic pipelining? With suitable examples and derivations describe how it speed up the computation?
2. Dynamic instruction scheduling reduce pipeline hazards and improve microprocessor performance? Justify
3. How can Cannon's algorithm and systolic arrays improve matrix multiplication efficiency in high-performance computing? Explain
4. What are some common use cases of vector databases in the field of data science and AI? Explain
5. What is vectorization? Explain why a vectorizing compiler is important in modern computing? Mention some challenges in vectorization and ways to overcome them.